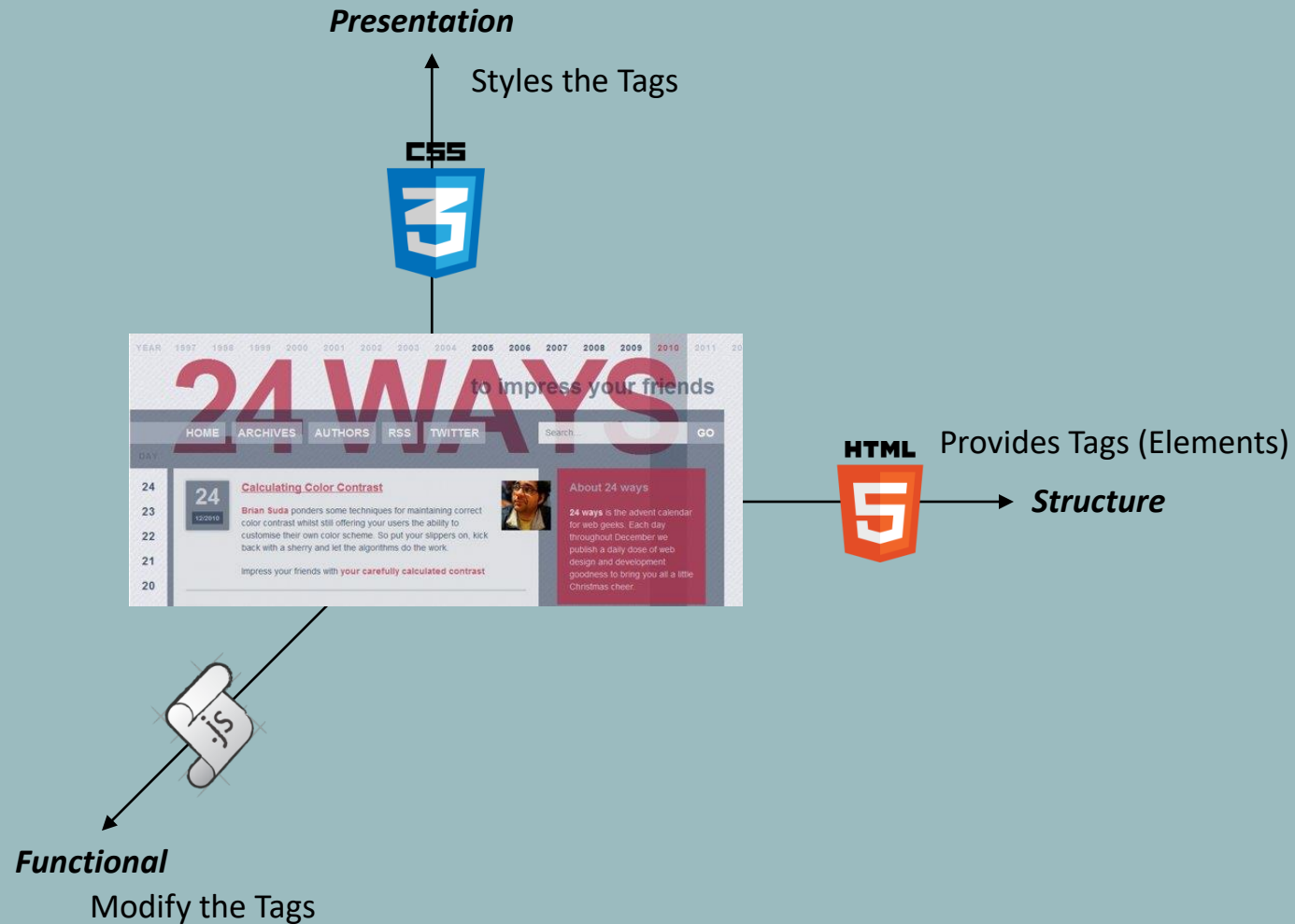


Web Development

>> JavaScript: Introduction

Separation of Concerns



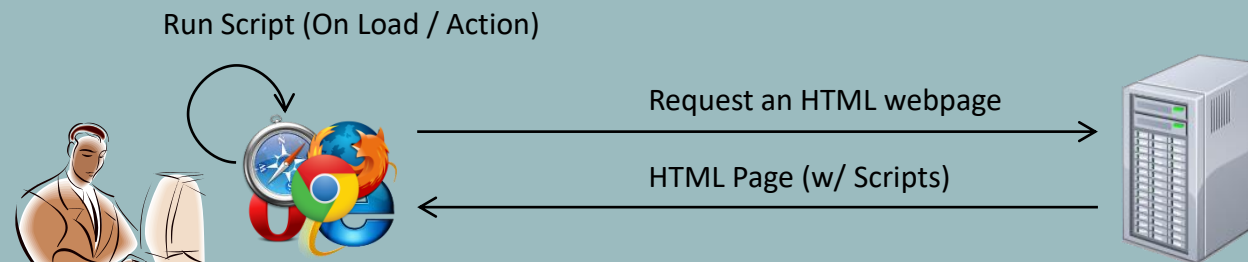
JavaScript - Introduction

- **JavaScript**

- Client-Side ***Scripting Language***

- It tells the browser to go do the work
 - Makes Webpages more interactive
 - JavaScript is **not** the same as Java
 - But has various similarities with the programming language

- **Security**



Rules:

1. JS **cannot read/write** files from/to the computer file system
2. JS **cannot execute** any other programs
3. JS **cannot** establish any **connection** to other computer, except to download a new HTML page or to send mail

Two functions

- **Popups or Alert Boxes**

- *alert("message")*

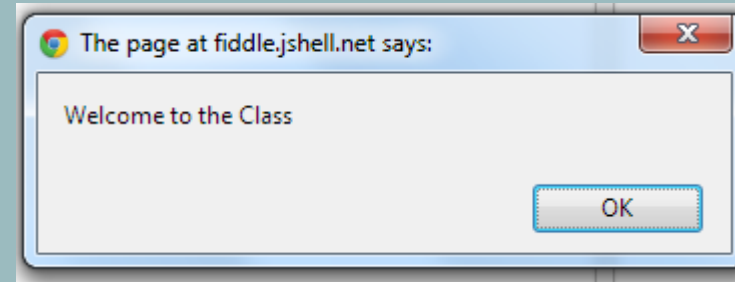
- **Write function**

- *document.write()*
- *Like the System.out.println() function in Java*

Pop-up Boxes in JS

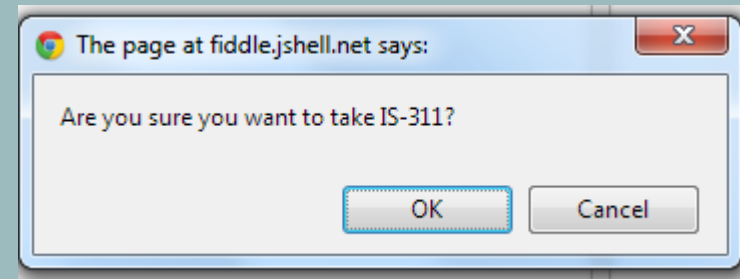
ALERT

`alert("Welcome to the Class");`



CONFIRM

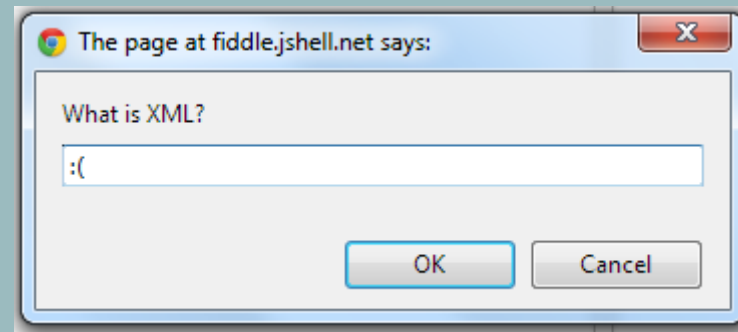
`confirm("Are you sure you want to take IS-311?");`



PROMPT

`prompt("What is XML?", ":(");`

Returns a value also



JavaScript Display Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using **innerHTML**.
- Writing into the HTML output using **document.write()**.
- Writing into an alert box, using **window.alert()**.
- Writing into the browser console, using **console.log()**

Writing into an HTML element, using **innerHTML**.

- To access an HTML element, JavaScript can use the **document.getElementById(id)** method.
- The **id** attribute defines the HTML element. The **innerHTML** property defines the HTML content:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p>My First Paragraph</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = 5 + 6;
```

```
</script>
```

```
</body>
```

```
</html>
```

Writing into the HTML output using **document.write()**.

- For testing purposes, it is convenient to use **document.write()**

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>My First Web Page</h1>
```

```
<p>My first paragraph.</p>
```

```
<button onclick="document.write(5 + 6)">Try it</button>
```

```
</body>
```

```
</html>
```


Writing into the browser console, using `console.log()`

- For debugging purposes, you can use the **`console.log()`** method to display data. (F12 in Chrome)

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<script>  
console.log(5 + 6);  
</script>
```

```
</body>  
</html>
```

Variables

Variables or **Identifiers** are named memory locations that hold data to be used throughout the code

Syntax

keyword value
var name = 23;
 variable name

Rules:

1. Case-sensitive
2. Cannot start with a number
3. Can contain letter, numbers & underscore

Note: *Must be declared before their use in the script*

TRY NOW

```
{ var x=23;  
  document.write(x);  
}
```

Types of Variables

- **Numbers:** Integers, Decimal Numbers, Negative Numbers
- **Text / String:** “Use quotations for values”
- **Boolean:** true / false
- **No Value:** null (*Empty Variable*) – Not same as a zero
- **Undefined:** *A variable without A value.*

You can declare many variables in one statement.

```
var person = "John Doe", carName = "Volvo", price = 200;
```

Strings in JavaScript

Quotes

You can use both **'single quotes'** and **"double quotes"**

For eg: **var str = "This is a sample string";**

Escape Characters

```
var x="I said "Hi" ";  
document.write(x);
```

Use backslash (\) to escape

Concatenation

Use the **“+”** operator to join two strings

```
var x="Web";  
var y= "Systems";  
document.write(x + " "+y);
```

JavaScript VAR, LET, CONST Key Differences

Keyword	Can Be Reassigned	Can Be Redeclared	Scope	Example
VAR	Yes	Yes	Function scope	var x = 10; x = 20; // Reassigned var x = 30; // Redeclared
LET	Yes	No	Block scope	let y = 5; y = 15; // Reassigned let y = 25; // Error: Cannot redeclare
CONST	No	No	Block scope	const z = 100; z = 200; // Error: Cannot reassign const z = 300; // Error: Cannot redeclare

JavaScript VAR, LET, CONST Key Differences

- VAR: Variables declared with **var** can be reassigned and redeclared. They are function-scoped, meaning they are only accessible within the function they are declared in.
- LET: Variables declared with **let** can be reassigned but not redeclared within the same scope. They are block-scoped, meaning they are only accessible within the block they are declared in.
- CONST: Variables declared with **const** cannot be reassigned or redeclared. They are also block-scoped. However, if the variable is an object or array, its properties or elements can be modified.

JavaScript VAR, LET, CONST Key Differences

- var Block Scope

```
function demoFunction() {  
  if (true) {  
    var y = 20;  
    console.log(y);    // Output: 20  
  }  
  console.log(y);    // Output: 20  
}
```

```
demoFunction();
```

JavaScript VAR, LET, CONST Key Differences

- let Block Scope

```
function demoFunction() {  
  if (true) {  
    let y = 20;  
    console.log(y);    // Output: 20  
  }  
  console.log(y);    // Error : y is not defined  
}
```

```
demoFunction();
```


Strings in JavaScript

- **Pre-defined Functions**

- **.length** – returns the length of the string
- **.split(a)** – splits the string (*later in Arrays*)- The split method returns the new array.
- **.substr(a, b)** – *extracts the substring between a and b*
- **toLowerCase()**
- **toUpperCase()**

- **Conversion**

- **Number to String:** Use **.toString()** function
- **String to Number:** Multiply by 1
 - Ensure that the string has only numbers (0-9) or use method **Number("3")**

TRY NOW

```
var a = 'Hello';  
var b = 'World';  
  
document.write(a+" "+b);  
  
document.write("<br\>");  
  
document.write(a.length);  
  
document.write(a.substr(2,4));
```

Operators

- “+”
 - Addition when both are numbers
 - Concatenation when any is a string
- Others
 - Minus(-), Multiply(*), Divide(/), Remainder(%)
 - Increment(++), Decrement(--)
 - Assignment (=)

Conditional Statement

- **If Statement**

- execute some code only if a specific condition is met.

- **Else If Statement**

- Various conditions that are checked one after another until the script finds a true condition

- **Else Statement**

- If none of the above conditions are met, this block of code is executed.

SYNTAX

keyword

```
keyword  
if (something is the case)  
{  
    more JavaScript commands  
}
```

<i>Larger than</i>	>
<i>Smaller than</i>	<
<i>Larger than or equal to</i>	>=
<i>Smaller than or equal to</i>	<=
<i>Equal to</i>	==
<i>Identical</i>	===
<i>Not equal to</i>	!=

example:

- 1. Write an if statement with the following condition: 10 is greater than 5.
- 2. If the condition is true, display "I did it!".

```
<!DOCTYPE html>
<html>
<body>
<p id="demo">Display the result here.</p>

<script>
if(10>5)
document.getElementById("demo").innerHTML="I Did It"

</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>

<p id="demo">Display the result here.</p>

<script>
var greeting;
var hour = new Date().getHours();

if (hour < 18) {
    greeting = "Good day";
} else {
    greeting = "Good evening";
}

document.getElementById("demo").innerHTML = greeting;
</script>

</body>
</html>
```

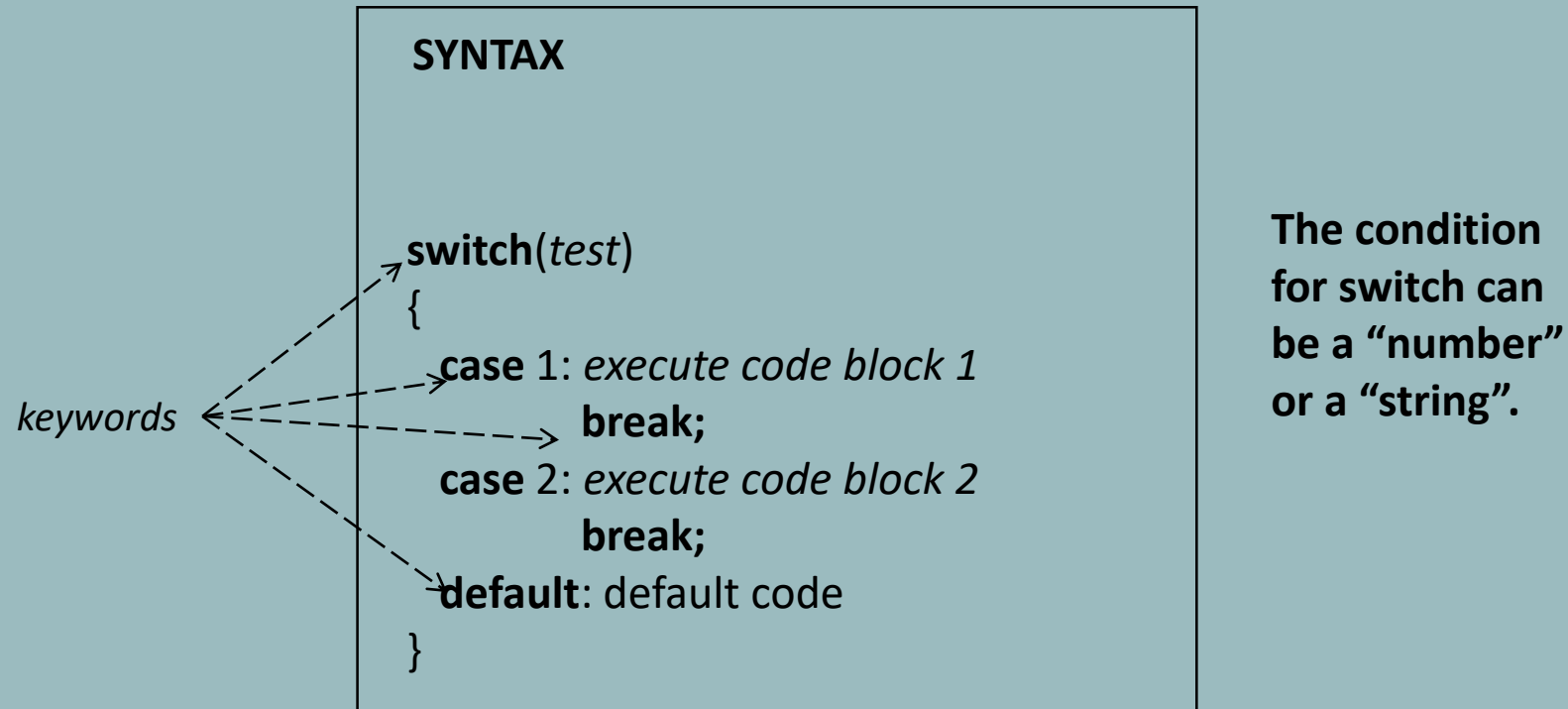
Exercise:

- The if/else statement does not work.
Try to fix it.

Conditional Statement

- **Switch Statement**

- Select one of many blocks of code to be executed



Example

```
switch (new Date().getDay()) {  
    case 0:  
        day = "Sunday";  
        break;  
    case 1:  
        day = "Monday";  
        break;  
    case 2:  
        day = "Tuesday";  
        break;  
    case 3:  
        day = "Wednesday";  
        break;  
    case 4:  
        day = "Thursday";  
        break;  
    case 5:  
        day = "Friday";  
        break;  
    case 6:  
        day = "Saturday";  
}
```


Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<input id="myInput" type="text" value="BMW">
```

```
<p id="demo"></p>
```

```
<script>
```

```
var text;
```

```
var favCar = document.getElementById("myInput").value;
```

```
switch(favCar) {
```

```
  case "BMW": text= "German car"; break;
```

```
  case "Ford": text= "American car"; break;
```

```
  case "Peugeot": text= "French car"; break;
```

```
  default: text="Unknown Car";
```

```
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body> </html>
```

Boolean Conditions

- **Combine Multiple conditions in the IF statement**

AND (&&)	True when both elements are true
OR ()	True when at least one of the elements is true
NOT (!)	Toggles a statement from true to false or from false to true

Looping Statement

- Initial Value; Test Condition; Update Value
- **For Statement**
 - execute some code repeatedly
- **While Statement**
 - Convenient when you want to loop until a condition changes
- **Do Statement**
 - Useful when you always want to execute the loop at least once

SYNTAX

```
keyword  
┌  
for (initialize; condition; update)  
{  
    more JavaScript commands  
}
```

```
Initialize outside  
keyword  
┌  
do  
{  
    more JavaScript commands  
    update inside  
} while (condition);
```

```
Initialize outside  
keyword  
┌  
while (condition)  
{  
    more JavaScript commands  
    update inside  
}
```

Examples

```
for (i = 0; i < 5; i++) {  
    text += "The number is " + i + "<br>";  
}
```

```
while (i < 10) {  
    text += "The number is " + i;  
    i++;  
}
```

```
do {  
    text += "The number is " + i;  
    i++;  
}  
while (i < 10);
```

Breaks - Continue

```
for (i = 0; i < 10; i++) {  
    if (i === 3) { break; }  
    text += "The number is " + i + "<br>";  
}
```

```
for (i = 0; i < 10; i++) {  
    if (i === 3) { continue; }  
    text += "The number is " + i + "<br>";  
}
```

SYNTAX

```
keyword  
└──  
function name()  
{  
  
}
```

Functions in JavaScript

- Mainly used for event-handling in Web Development
- Can also be called from other functions (Reuse)

TRY NOW

```
{  
  function test()  
  {  
    alert("Tested");  
  }  
}
```

Tip: Make it work by calling the function

Functions in JavaScript

```
function name(parameters)
{
    return b;
}
```

- You can use as many parameters as you like
- Can also return values (numbers, strings, boolean)
- Use **return** statement

TRY NOW

```
function toCelsius(fahrenheit) {
    return (5/9) * (fahrenheit-32);
}
document.getElementById("demo").innerHTML =
toCelsius(10);
```

Function Call

```
var x = toCelsius(77);
var text = "The temperature is " + x + "
Celsius";
```

```
{
    function test(x)
    {
        alert(x);
    }
}
```

Functions Exercises

- Write a JavaScript function that reverses a number.

Sample Data and output:

Example x = 12345;

Expected Output: 54321

Functions Exercises

- Write a JavaScript that used the onClick Event to reverse a number given in textfield 1 into another textfiled .

12345

54321

Reverse

Functions Exercises

- Create a function that finds the maximum range of a triangle's third edge, where the side lengths are all integers.

- **Examples**

- `nextEdge(8, 10) → 17`
- `nextEdge(5, 7) → 11`
- `nextEdge(9, 2) → 10`

Assignments

Notes

$(\text{side1} + \text{side2}) - 1 = \text{maximum range of third edge.}$

The side lengths of the triangle are positive integers.

Don't forget to return the result.

Functions Exercises

- Write a JavaScript function that check if the words sequence are read the same backward as forward.
- Example : MADAM , ABCDCBA

Assignments

Functions Exercises

- Write a JavaScript function that returns a passed string with letters in alphabetical order

Example string: 'webmaster'

Expected Output: 'abeemrstw'

Assignments